# <u>Machine Learning & Deep Learning</u>

## Experiment 10

**Name: Abhishek Vishwakarma**
**Class: D15C**
**Roll No: 47**

**AIM:** Explore and implement Autoencoders for image denoising

## 1. Dataset Source

- **Dataset Name:** Fashion-MNIST
- **Source:** Zalando Research - Fashion-MNIST (Available directly via the TensorFlow/Keras API).

## 2. Dataset Description

Fashion-MNIST is a dataset of Zalando's article images, serving as a drop-in replacement for the original MNIST dataset for benchmarking machine learning algorithms.

- **Size:** 60,000 training examples and 10,000 testing examples.
- **Dimensions:** 28x28 pixels, Grayscale (1 color channel).
- **Target Variable:** The dataset has 10 class labels (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot).

## 3. Mathematical Formulation of the Algorithm

An Autoencoder is an unsupervised neural network that learns to compress data into a lower-dimensional space and then reconstruct it. A **Denoising Autoencoder** is trained specifically to ignore noise.

**A. Noise Addition:**

We first corrupt the original clean input x by adding random noise to create a noisy input ẋ :

$$\tilde{x} = x + \epsilon \quad \text{where} \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

**B. Encoder:**

The encoder function f maps the noisy input ẋ to a compressed hidden representation (the latent space or bottleneck) h:

$$h = f_\theta(\tilde{x}) = \sigma(W\tilde{x} + b)$$

**C. Decoder:**

The decoder function g maps the hidden representation h back to the original dimensions to produce a reconstructed image ẋ :

$$\hat{x} = g_\phi(h) = \sigma(W'h + b')$$

**D. Objective Function (Loss):**

The network learns the parameters θ and ϕ by minimizing the reconstruction error between the **reconstructed image** ẋ and the **original clean image x** (not the noisy one). The Mean Squared Error (MSE) is commonly used:

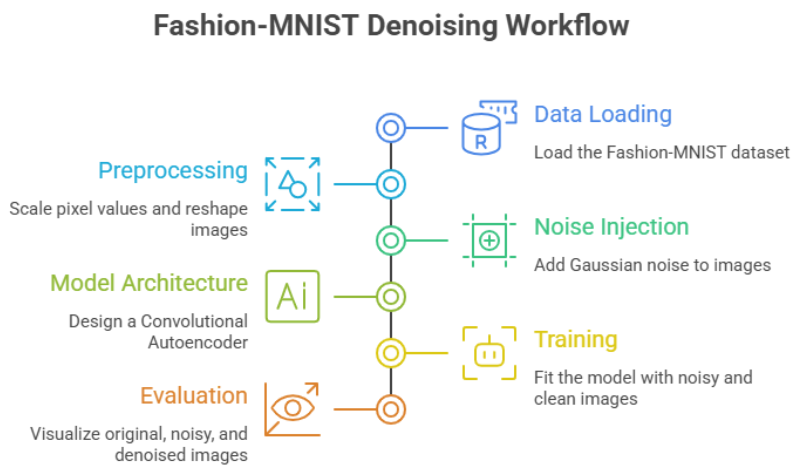
$$L(x, \hat{x}) = \frac{1}{N} \sum_{i=1}^{N} (x_i - \hat{x}_i)^2$$

## 4. Algorithm Limitations

- **Blurry Reconstructions:** Because autoencoders typically use MSE as a loss function, they tend to average out uncertainties, leading to slightly blurry reconstructed images compared to the sharp originals.
- **Data Specificity:** An autoencoder trained to denoise clothing (Fashion-MNIST) will fail completely if you ask it to denoise human faces. It only learns the specific data distribution it was trained on.
- **Bottleneck Sizing:** If the latent space (bottleneck) is too large, the network might just learn the identity function (copying the noise directly). If it is too small, it loses too much critical image information.

## 5. Methodology / Workflow

1. **Data Loading:** Load the Fashion-MNIST dataset.
2. **Preprocessing:**
    - Scale pixel values to [0, 1].
    - Reshape images to (28, 28, 1) to add the channel dimension required by Convolutional layers.
3. **Noise Injection:** Add Gaussian noise to both the training and testing sets. Clip the resulting values to ensure they remain between 0 and 1.
4. **Model Architecture (Convolutional Autoencoder):**
    - **Encoder:** Convolutional layers followed by Max Pooling layers to reduce spatial dimensions and extract features.
    - **Decoder:** Convolutional layers followed by UpSampling layers to rebuild the spatial dimensions back to 28 × 28.
5. **Training:** Fit the model using x = noisy_images and y = clean_images.
6. **Evaluation:** Visualize the Original, Noisy, and Denoised images side-by-side to qualitatively assess performance.




## 6. Performance Analysis

- **Metric:** Validation Loss (MSE).
- **Analysis:** Unlike classification tasks where we look at accuracy, autoencoders are evaluated visually and by their reconstruction loss. A decreasing MSE indicates the model is successfully learning the underlying structure of the clothing items while discarding the random noise.

## 7. Hyperparameter Tuning

- **Noise Factor:** Controls how much noise is added to the training data. Higher noise makes the task harder and forces the model to learn more robust features.
- **Latent Dimension (Filters/Layers):** Adjusting the number of filters in the deepest layer changes the bottleneck size.
- **Optimizer:** Adam is standard, but tuning its learning rate can speed up convergence.

**Code:**

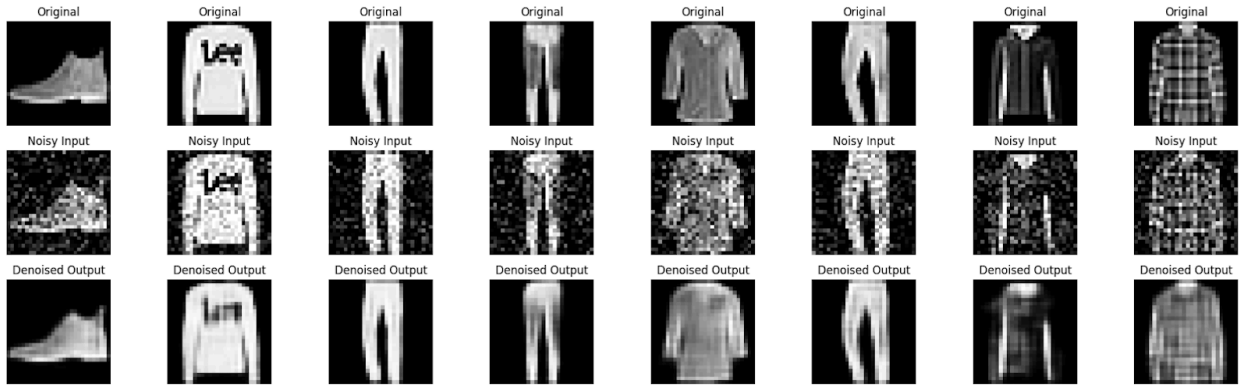
```
# ═══════════════════════════════════════════════════════════════
# EXPERIMENT 10: AUTOENCODERS FOR IMAGE DENOISING
# ═══════════════════════════════════════════════════════════════

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import fashion_mnist

print("--- EXPERIMENT 10: CONVOLUTIONAL DENOISING AUTOENCODER ---")

# 1. Load Dataset
# We only need the images, not the labels (unsupervised learning)
(x_train, _), (x_test, _) = fashion_mnist.load_data()

# 2. Preprocessing
# Normalize to [0, 1]
```

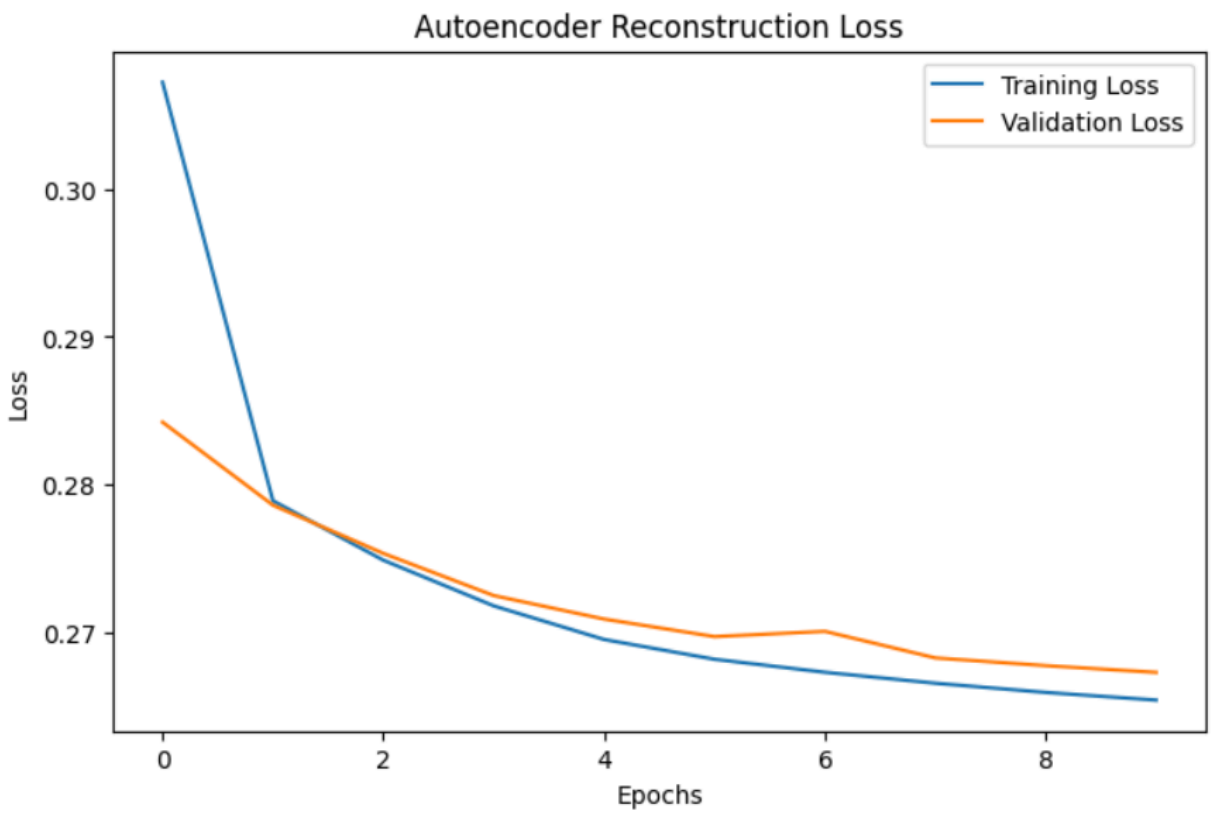
```
x_train = x_train.astype('float32') /
255.
x_test = x_test.astype('float32') / 255.

# Reshape for Convolutional Layers
(adding the channel dimension)
x_train = np.reshape(x_train,
(len(x_train), 28, 28, 1))
x_test = np.reshape(x_test,
(len(x_test), 28, 28, 1))

print(f"Training data shape:
{x_train.shape}")
print(f"Testing data shape:
{x_test.shape}")

# 3. Add Artificial Noise
noise_factor = 0.2
x_train_noisy = x_train + noise_factor *
tf.random.normal(shape=x_train.shape)
x_test_noisy = x_test + noise_factor *
tf.random.normal(shape=x_test.shape)

# Clip values to stay within valid image
range [0, 1]
x_train_noisy =
tf.clip_by_value(x_train_noisy,
clip_value_min=0., clip_value_max=1.)
x_test_noisy =
tf.clip_by_value(x_test_noisy,
clip_value_min=0., clip_value_max=1.)

# 4. Build the Convolutional Autoencoder
Architecture
autoencoder = models.Sequential()

# ENCODER
# Input is 28×28×1
autoencoder.add(layers.Input(shape=(28,
28, 1)))
autoencoder.add(layers.Conv2D(32, (3,
3), activation='relu', padding='same'))
autoencoder.add(layers.MaxPooling2D((2,
2), padding='same')) # Reduces to 14×14
autoencoder.add(layers.Conv2D(32, (3,
3), activation='relu', padding='same'))
autoencoder.add(layers.MaxPooling2D((2,
2), padding='same')) # Reduces to 7×7
(Bottleneck)

# DECODER
autoencoder.add(layers.Conv2D(32, (3,
3), activation='relu', padding='same'))
autoencoder.add(layers.UpSampling2D((2,
2))) # Rebuilds to 14×14
autoencoder.add(layers.Conv2D(32, (3,
3), activation='relu', padding='same'))
autoencoder.add(layers.UpSampling2D((2,
2))) # Rebuilds to 28×28
# Final layer uses Sigmoid to ensure
output is between 0 and 1
autoencoder.add(layers.Conv2D(1, (3, 3),
activation='sigmoid', padding='same'))

autoencoder.summary()

# 5. Compile the Model
# We use binary_crossentropy or mse for
reconstruction
autoencoder.compile(optimizer='adam',
loss='binary_crossentropy')

# 6. Train the Model
print("\nTraining Autoencoder (mapping
noisy images to clean images)...")
history = autoencoder.fit(x_train_noisy,
x_train,
                          epochs=10,

batch_size=128,
                          shuffle=True,

validation_data=(x_test_noisy, x_test))

# 7. Evaluate and Visualize
# Generate denoised images from the
noisy test set
decoded_imgs =
autoencoder.predict(x_test_noisy)

# Plotting Original, Noisy, and Denoised
Images
n = 8  # Number of images to display
```

```
plt.figure(figsize=(20, 6))
for i in range(n):
    # Display Original Clean Image
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(tf.squeeze(x_test[i]),
cmap='gray')
    plt.title("Original")
    plt.axis('off')

    # Display Noisy Image
    ax = plt.subplot(3, n, i + 1 + n)

plt.imshow(tf.squeeze(x_test_noisy[i]),
cmap='gray')
    plt.title("Noisy Input")
    plt.axis('off')

    # Display Denoised Image
    ax = plt.subplot(3, n, i + 1 + 2 *
n)

plt.imshow(tf.squeeze(decoded_imgs[i]),
cmap='gray')
    plt.title("Denoised Output")
    plt.axis('off')

plt.tight_layout()
plt.show()

# Visualize Training Loss
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'],
label='Training Loss')
plt.plot(history.history['val_loss'],
label='Validation Loss')
plt.title('Autoencoder Reconstruction
Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

**Output:**

```
Training Autoencoder (mapping noisy images to clean images)...
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 134s 280ms/step - loss: 0.3599 - val_loss: 0.2842
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 144s 285ms/step - loss: 0.2797 - val_loss: 0.2786
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 141s 282ms/step - loss: 0.2756 - val_loss: 0.2753
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 140s 277ms/step - loss: 0.2726 - val_loss: 0.2725
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 131s 280ms/step - loss: 0.2698 - val_loss: 0.2709
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 131s 280ms/step - loss: 0.2688 - val_loss: 0.2697
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 131s 280ms/step - loss: 0.2673 - val_loss: 0.2701
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 144s 284ms/step - loss: 0.2668 - val_loss: 0.2683
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 133s 283ms/step - loss: 0.2661 - val_loss: 0.2677
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 132s 282ms/step - loss: 0.2659 - val_loss: 0.2673
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 21ms/step
```




## 8. Conclusion

In this experiment, we constructed a Convolutional Autoencoder to perform image denoising on the Fashion-MNIST dataset.

- Result: The UpSampling layers then successfully reconstructed the images from the compressed features. As seen in the visual output, the network effectively removed the static/grainy noise, leaving behind a smooth, highly recognizable reconstruction of the original item.